# Dynamical Systems Submission

Choomno Moos*

November 18, 2020

---

## 1 Implementation of a Root Finder via Multivariate Newton's Method

We implement a multivariate Newton's method to solve for one root of a system of nonlinear equations given by

$$f(x) = \begin{pmatrix} x_1 + 2x_2 - 3x_3 + 3 \\ x_1^2 + 4x_2^2 - 4x_3 \\ x_1^2 - x_2^2 - x_3^2 \end{pmatrix} = 0,$$

where $x = (x_1, x_2, x_3)$. Geometrically, this represents the intersection of a plane, a paraboloid, and a double cone. A bit of 3D plotting suggests that there are four roots; this is especially apparent if we transform into the plane of the first equation, where we would see the problem reduced to finding the intersections of two conic sections.

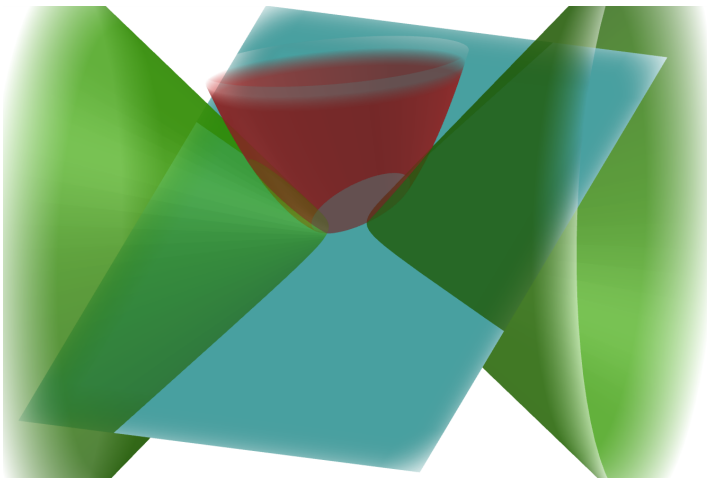


Figure 1: A plot of the system of equations in question done in Geogebra. If you look carefully you can see that there are 4 roots.

*email: choom@pdx.edu

The multivariate Newton's method is given by

$$x_{n+1} = x_n - J^{-1}(x_n)f(x_n), \tag{1}$$

where $J$ is the Jacobian matrix of $f$, so our first step is to compute the Jacobian matrix $J$ of $f(x)$,

$$J = \begin{pmatrix} 1 & 2 & -2 \\ 2x_1 & 8x_2 & -4 \\ 3x_1^2 & -4x_2 & 2x_3 \end{pmatrix}.$$

We now make a slight alteration to the method given by equation (1). While calculating the inverse $J^{-1}$ is easy enough in this case, in general it is computationally prohibitive. So let $\Delta x_n = x_{n+1} - x_n$, and rewrite equation (1) as

$$J(x_n)\Delta x_n = -f(x_n).$$

We can then solve for $\Delta x_n$ using Gaussian elimination. We set out initial guess to be $x_0 = (-1, 1, 1)$, and implement the procedure in Python. The code is given below

```
# Choomno Moos
# Oct 29, 2020

import numpy as np
import scipy.linalg as la

x_0 = [-1,1,1]

def f(x_1,x_2,x_3):
    return np.array([x_1 + 2*x_2 - 3*x_3 + 3,
                     x_1**2 + 4*x_2**2 - 4*x_3,
                     x_1**2 - x_2**2 - x_3**2])

def J(x_1,x_2,x_3):
    return np.array([[1,2,-3],
                     [2*x_1,8*x_2,-4],
                     [2*x_1,-2*x_2,-2*x_3]])

def iterate_newton(x_old):
    Delta_x_n = la.solve(J(x_old[0],x_old[1],x_old[2]),
                         -f(x_old[0],x_old[1],x_old[2]))
    x_new = x_old + Delta_x_n
    return x_new

x = x_0
for i in range(10):
    print(x)
    x_old = x
    x = iterate_newton(x)
    if (x[0] == x_old[0] and x[1] == x_old[1] and x[2] == x_old[2]):
        break
```

The program prints the following values for $x_n$:

```
[-1,  1,  1]
[-1.39473684   0.81578947   1.07894737]
[-1.34353582   0.79494319   1.08211686]
[-1.34267782   0.7946967    1.08223852]
[-1.3426776    0.79469667   1.08223858]
[-1.3426776    0.79469667   1.08223858]
```

Evaluating the function at the value of our final iterate yields a vector which is very, very close to zero. Given the constraints of floating point arithmetic, it is safe to say that our program has proven effective. Finding the other three solutions is only a matter of changing the initial value $x_0$ to be in the basin of attraction of each respective root.

# 2 Computer Generated Fractals

It's not hard to directly apply Newton's method to see approximately what the Julia set associated to $f : z \to z^3 - 1$ looks like. Below are two such images, they each took less than 10 minutes generate.

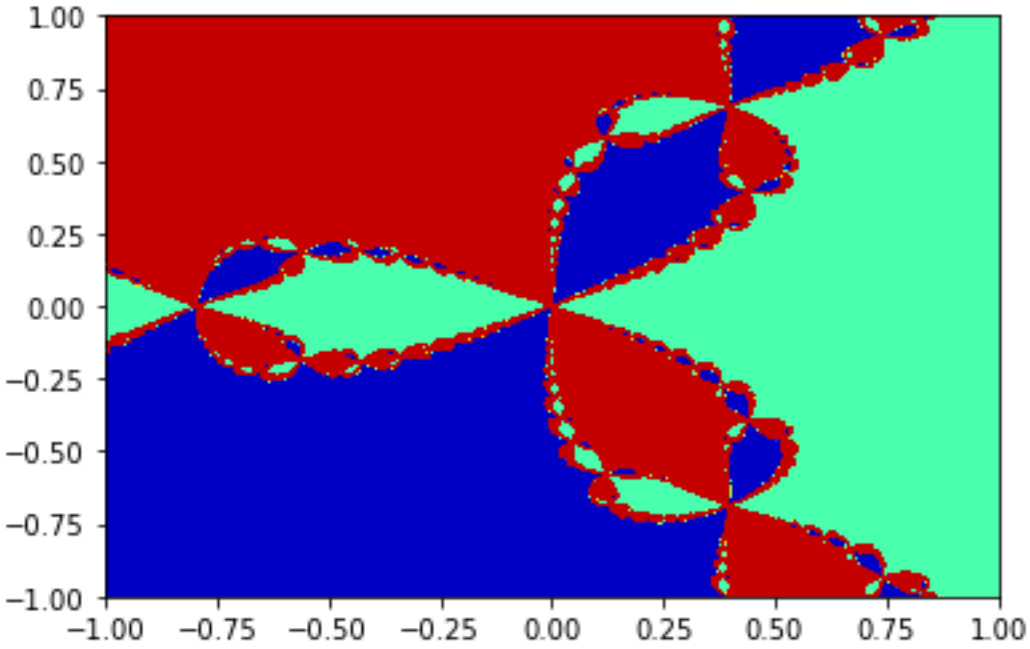



Figure 2: Newton's method applied to $f : z \to z^3 - 1$

Here is another picture for $f : z \to z^4 + 1$.

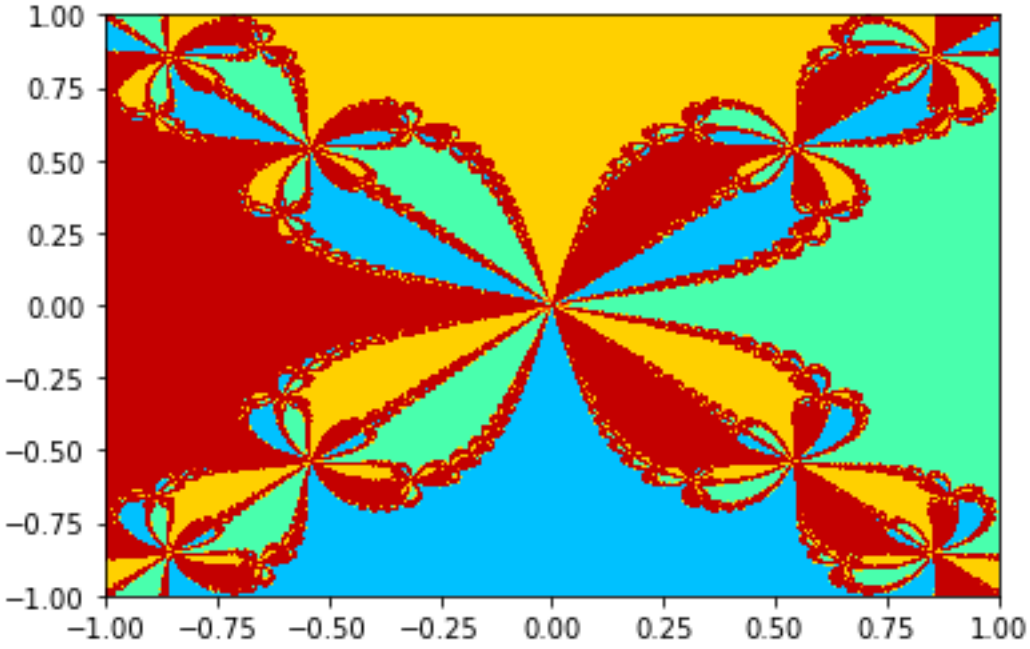



Figure 3: Newton's method applied to $f : z \to z^4 - 1$

The simplicity of my program have made the pictures look very crude; the divergent points aren't distinguished, so you can't actually see the Julia set (it is defined to lie on the fractal boundary, where orbits exhibit sensitive dependence), and there is no shading to indicate how many iterations were required before the point converged.

Below is the Python code. Notice that it relies on large matrix computations (The matrix $Z$ defined on line 9 in the code below is an $n \times n$ complex-valued matrix, and we set $n = 10,000$!). This makes it memory intensive and thus slow. Still, I think that it's very nice in showing where Newton's method is being used.

```
1  # Choomno Moos
2  # Oct 29, 2020
3
4  from PIL import Image
5
6  w = 1920
7  h = 1080
8
9  julia = Image.new("RGB", (w, h), "white")
10
11 pix = julia.load()
12
13 c = -0.7269 + 0.1889j
14 max_iterations = 255
15
16 def f(z):
17     return z**2 + c
18
19 # The reason for z being defined as seen here is to center the picture.
20 for x in range(w):
21     for y in range(h):
22         z = 1.5*(x - 0.5*w)/(0.3*w) + 1.0*(y - 0.5*h)/(0.3*h)*1j
23         i = max_iterations
24         while (abs(z) <= 2 and i > 1):
25             z = f(z)
26             i -= 1
27         pix[x,y] = (i << 20) + (i << 12) + i*8
28 julia.save("julia.png")
```

It turns out that, in general, the behavior of the family of complex functions given by $z \to z^n + c$ is very interesting and complicated, and can often yield pleasing fractal images. For now, we narrow our attention to the orbit under iteration of the family of complex functions $f_c(z) = z^2 + c$ where $z_0 = 0$. We prove a few simple facts.

**Proposition.** *(Escape radius) for $|c| > 2$, the sequence $f_c^{\circ n}(z) = z^2 + c$ is not bounded. The sequence resulting from taking the modulus under iteration of $f$ with $|c| > 2$, $\{|f_{|c|>2}^{\circ n}|\}_n$, increases monotonically to infinity.*

*Proof.* Assume $|c| > 2$. We proceed by induction. Observe that when $n = 1$,

$$|f^{\circ 2}(0)| = |c^2 + c| \geqslant |c^2| - |c| = |c|(|c| - 1) > |c| = f_c(0)$$

So assume that for some $n$, $f_c^{\circ n} > c$. Then the quotient of the $n$th and $(n+1)$-th iterates is

$$\frac{f_c^{\circ n+1}(0)}{f_c^{\circ n}} = \frac{(f_c^{\circ n}(0))^2}{f_c^{\circ n}} \geqslant \frac{|f_c^n(0)|^2 - |c|}{|f_c^n(0)|} > |c| - 1 > 1.$$

It follows that $\{|f_{c>2}^{\circ n}|\}_n$ is unbounded and increases monotonically. Thus we have shown the desired. ■

**Definition 2.0.1.** Let $f(z) = z^2 + c$. The **Mandelbrot space** $\mathcal{M}$ is a parameter space given by the Mandelbrot set of parameters for which the corresponding sequences under iteration of $f$ is bounded

$$Mandelbrot = \{c \in \mathbb{C} : |f^{\circ n}(0)| \leqslant 2 \text{ for all } n \geqslant 0\}$$

with the subspace topology from Euclidean 2-space ($\mathbb{C} \cong \mathbb{E}^2$).

**Proposition.** *The Mandelbrot Space is a closed subset of Euclidean space.*

*Proof.* We prove that the complement of the Mandelbrot space $\mathbb{C} \setminus \mathcal{M}$ is open. Take $c \in \mathbb{C} \setminus \mathcal{M}$. Then by Proposition 1, for every real $r > 0$ there exists an $n$ such that $|f_c^n(0)| > r$. Fix one such $n$ with $r = 2$. Then call $\varepsilon = |f_c^{\circ n}(0)| - 2$, and observe that the $\varepsilon$-neighborhood $U$ of $|f_c^{\circ n}(0)|$ is clearly an open neighborhood. Further, $f_c^{\circ n}$ is continuous, so the preimage $(f_c^{\circ n-1})^{-1}(U)$ is an open neighborhood of $c \in \mathbb{C}$. Further, we constructed $U$ such that it cannot intersect with the Mandelbrot set. Hence $\mathbb{C} \setminus \mathcal{M}$ is open, so $\mathcal{M}$ is closed. ■

**Corollary 2.0.1.** *The Mandelbrot space is compact, since it is a closed and bounded subset of a Euclidean space. This is a consequence of the Heine-Borel theorem.*

We now endeavor to take create an image of the Mandelbrot set. We iterate for parameters such that $|c| < 2$ and terminate when either $|z_n| > 2$ or when we reach an iteration limit. This time we shade our picture based on the number of iterations before divergence (black is bounded, and hence indicates containment in the Mandelbrot set). This method is called the *escape time algorithm*.

```
1  # Choomno Moos
2  # Oct 29, 2020
3
4  from PIL import Image
5
6  w = 1920
7  h = 1080
8
9  mandelbrot = Image.new("RGB", (w, h), "white")
10
11 pix = mandelbrot.load()
12
13 max_iterations = 255
```

```
14
15 def f(z):
16     return z**2 + c
17
18 # The reason for c being defined as seen here is to center the picture.
19 # It is also possible to zoom in and shift the picture by changing c.
20 for x in range(w):
21     for y in range(h):
22         c = 2.0*(x - 0.7*w)/(0.5*w) + 1.5*(y - h/2)/(0.5*h)*1j
23         z = 0
24         i = max_iterations
25         while (abs(z) <= 2 and i > 1):
26             z = f(z)
27             i -= 1
28         pix[x,y] = (i << 20) + (i << 12) + i*8
29 mandelbrot.save("mandelbrot.png")
```

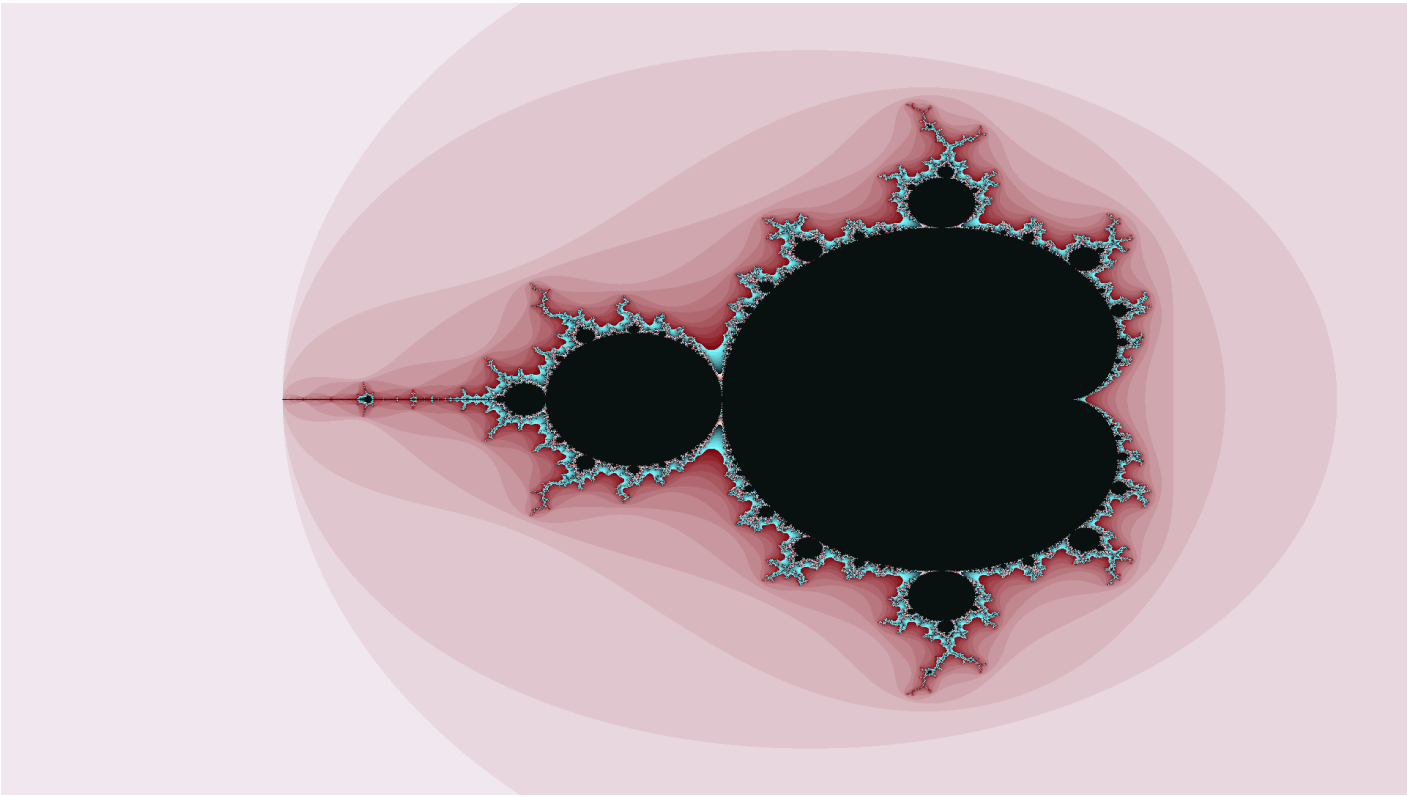


Figure 4: The Mandelbrot set image generated by our program. It's beautiful!

There are many interesting things about the Mandelbrot set. One that is particularly relevant is that there is a bijection of the Mandelbrot set's intersection with the real axis to the logistic family. To see this, first take the logistic family

$$x_{n+1} = \mu x(1 - x),$$

and, "completing the sqaure", rewrite it as

$$x_{n+1} = -\mu\left(x_n - \frac{1}{2}\right)^2 + \frac{\mu}{4}.$$

Then, apply the change of variables

$$z_n = i\sqrt{\mu}\left(x_n - \frac{1}{2}\right), \quad \text{and} \quad c = \frac{\mu}{4},$$

so that $x_{n+1} = z_{n+1} = z_n^2 + c$. Let us now find the values of $c$ for which the Mandelbrot set intersects the real line. The number $-2$ is in the Mandelbrot set by Proposition 1, and it is pretty obvious that $-2$ is real under iteration of $z \to z^2 + c$, so it must be the left bound. Now consider $c = 1/4$. If the iteration is to have a limit, it must be that $z = z^2 + c$, which has the solution

$$z = \frac{1 + \sqrt{1 - 4c}}{2}.$$

Hence $z$ becomes imaginary at any $c > 1/4$, so the interval of intersection is $[-2, 1/4]$.

It turns out that, the Mandelbrot set exhibits period doubling in a similar way to the logistic map. Starting from the main cardioid, each bulb to the left doubles and eventually undergoes chaos. It is easy to make out corresponding features, for example the mini-brot to the left is the notable 3-period region of stability you can see on the bifurcation diagram of the logistic map.

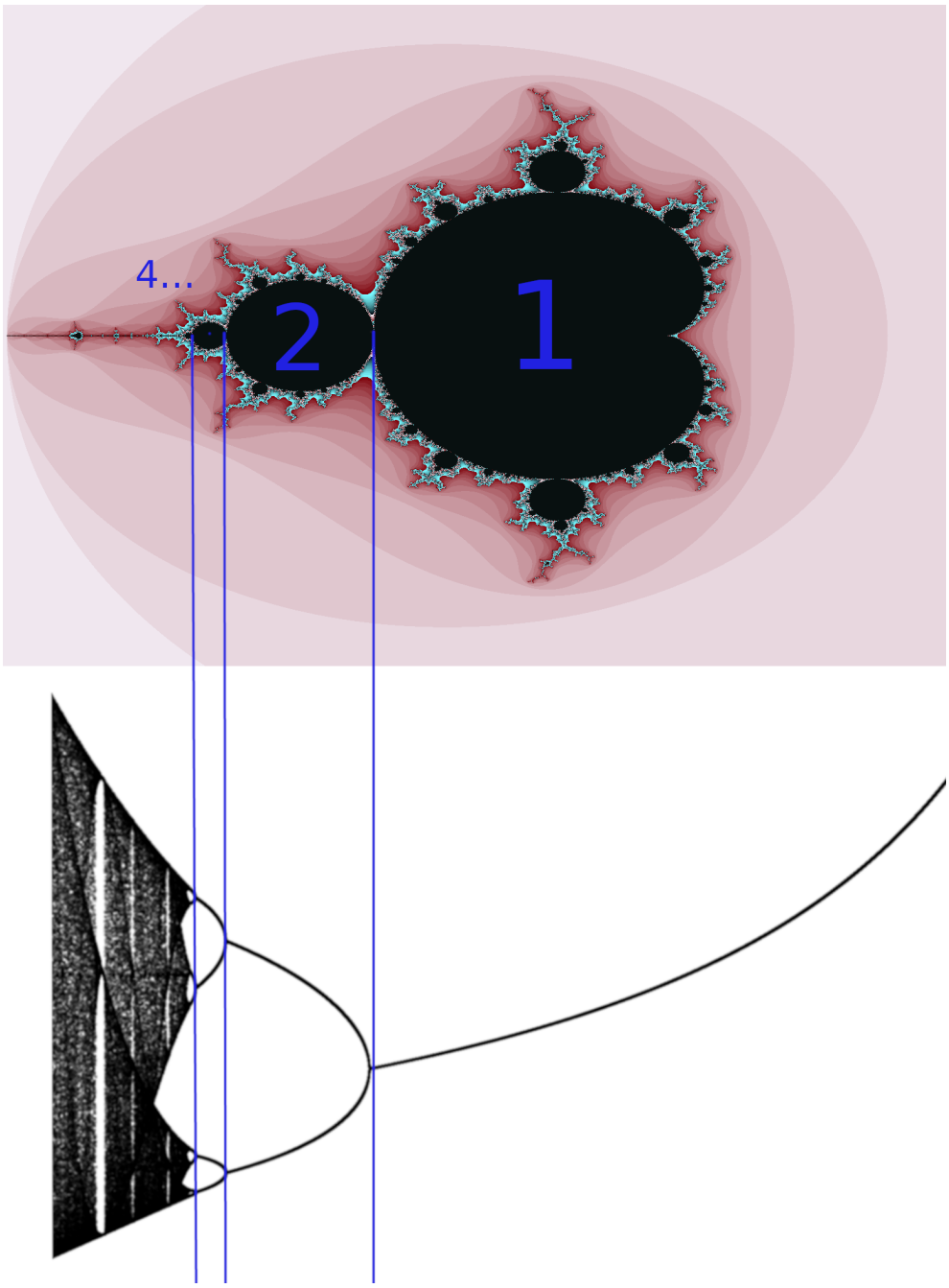



Figure 5: The logistic map corresponds to $c \in [-2, 1/4]$ in the Mandelbrot set.

Now let's see what happens when we fix a point $c$ in the Mandelbrot set and vary $z$ instead. Very little alteration of the code used to generate the Mandelbrot set yields a program to generate these sets.

When $|c| \leqslant 2$, we obtain what are called filled Julia sets. Like the Mandelbrot set, these sets exhibit fractal behavior. So in addition to being a fractal itself, each point in the Mandelbrot set corresponds to a yet another fractal!

We tour some of the popular destinations; the $c$ value is provided, and some of these locations are named, in which case we indicate this. Each of these sets exhibit complicated behavior, like period multiplication, chaos, and interesting dynamics between the "bulbs" of bounded points.

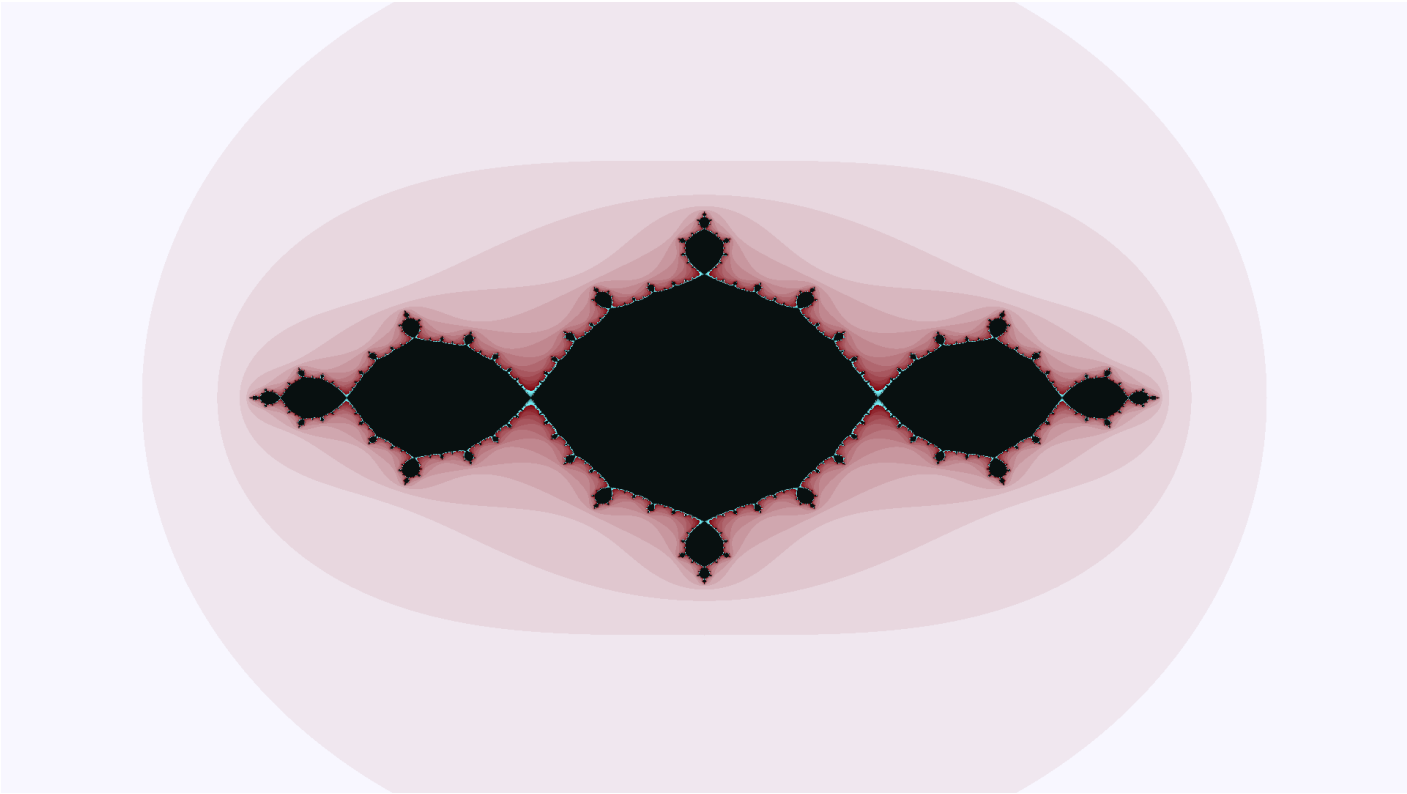


Figure 6: "Basilica", $c = -1$.

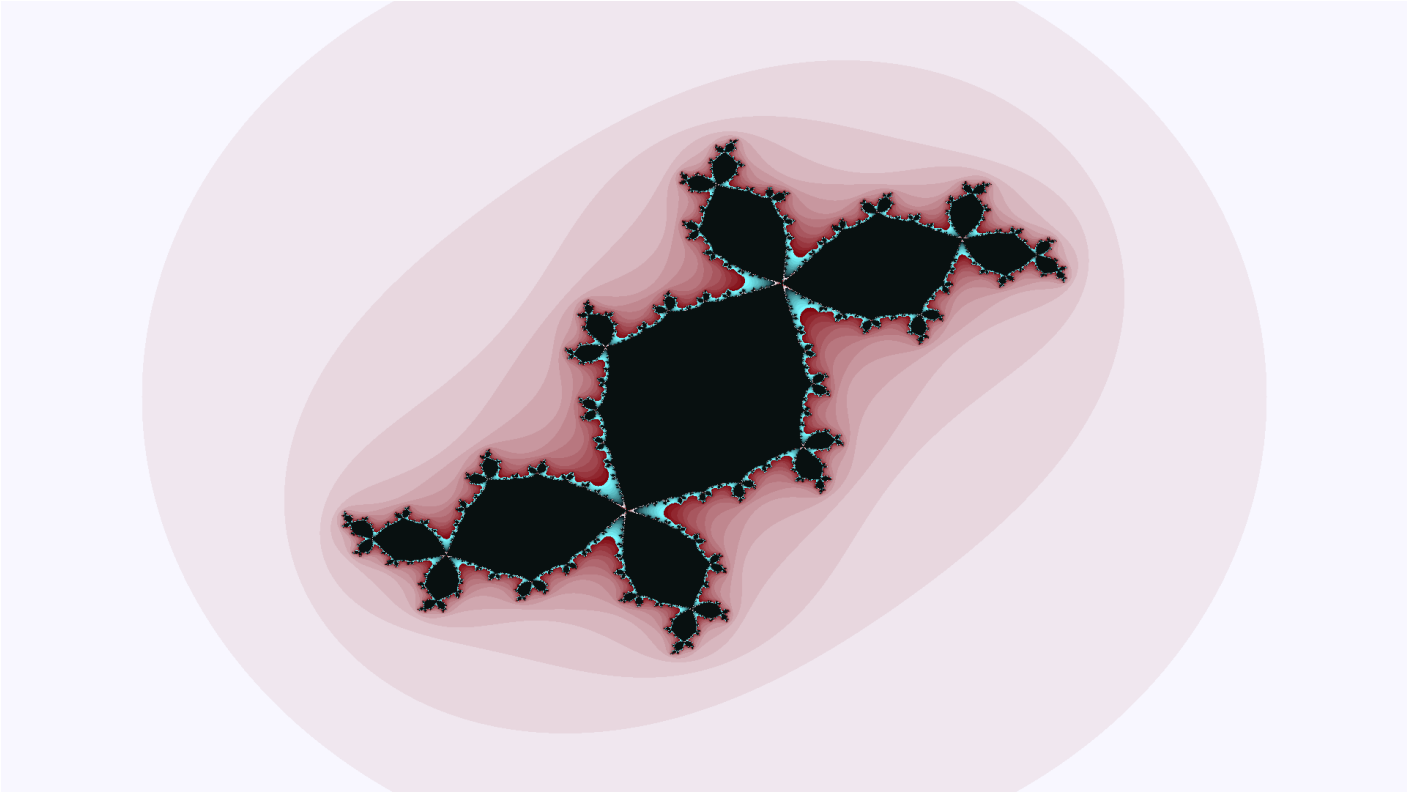


Figure 7: "Douady's Rabbit", $c = -0.122 + 0.745i$.

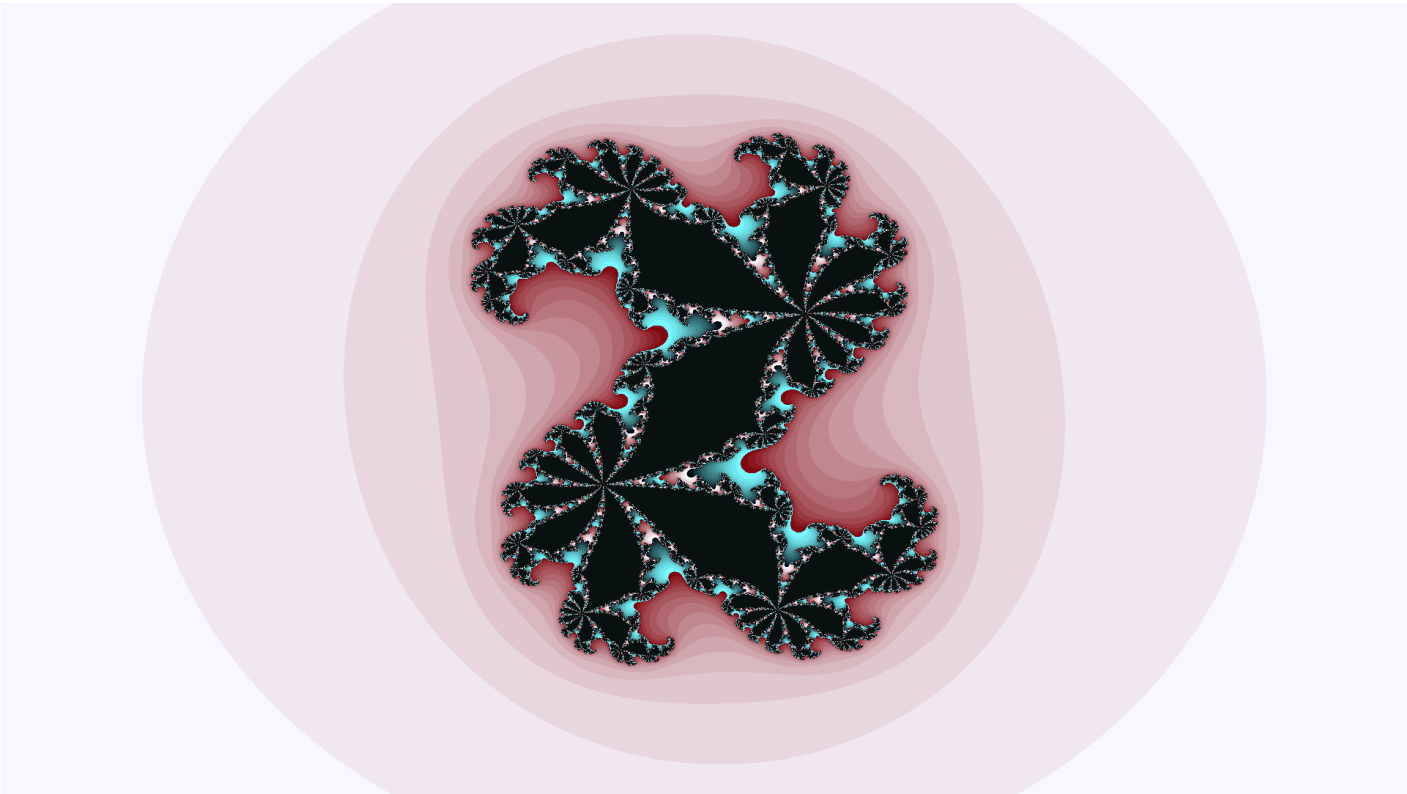


Figure 8: "Dragon", $c = 0.360283 + 0.100376i$.

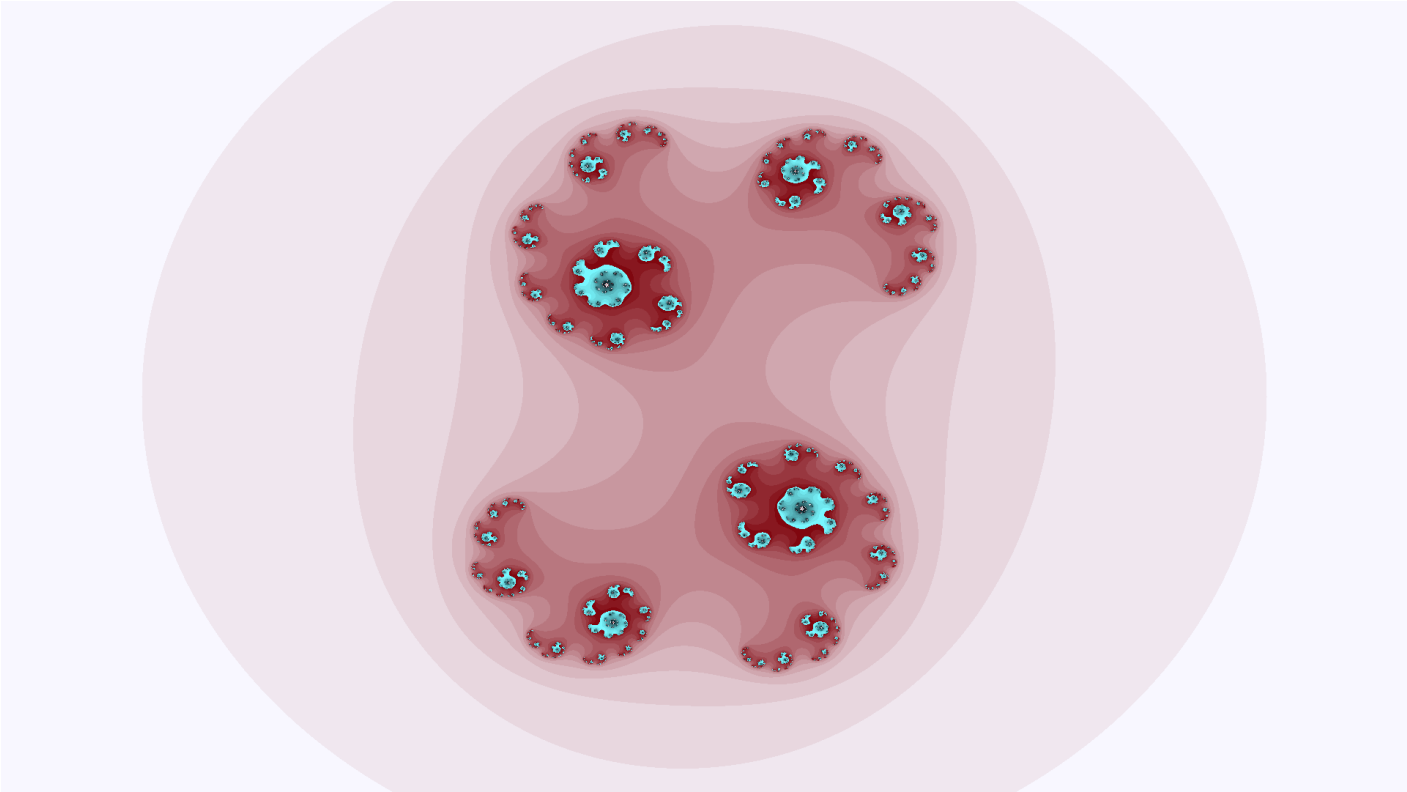


Figure 9: To maintain neutrality, we present one picture that's not so stunning. $c = 0.45 + 0.1428i$. I believe this one is still topologically connected!

We have been looking at the sets generated under iteration of a quadratic family of complex-valued functions, however they are part of a much more general family. We conclude by formally introducing Julia sets.

**Definition 2.0.2.** Call a complex function

$$f(z) = \frac{p(z)}{q(z)}$$

rational if $p$ and $q$ are complex polynomials. Let $f : \overline{\mathbb{C}} \to \overline{\mathbb{C}}$ be a non-constant holomorphic rational function from the Riemann sphere to itself where $p$ and $q$ have no common divisors. The **filled Julia set** corresponding to $f$ is the set of points $z$ which are bounded under iteration of $f$.

**Definition 2.0.3.** The **Julia set** itself is the boundary of the filled Julia set, around which there is sensitive dependence on initial conditions. This means for a point $z$ in the Julia set and any $\delta > 0$, there is a $w$ in any neighborhood around $z$ where $z$ and $w$ are at least $\delta$ apart under iteration with $f$.

*Remark.* By holomorphic we mean that $f$ is complex analytic, or complex differentiable. The Riemann sphere is the one-point compactification of the complex plane, $\mathbb{C} \cup \{\infty\}$ (or simply the extended complex numbers, if you prefer). Also note that there are several equivalent definitions of the Julia set, so this one is by no means definitive.

# 3 Numerically Finding Superstable of Orbits of the Logistic Map $x \to \mu x(1-x)$ for $\mu \in [3.5, 3.8]$

Note that it is sufficient to find the unique superstable points of each period in the period-doubling region of the logistic family, since after chaos commences, there are no stable orbits of period $2^n$.

I tried to follow Sternberg's sketch for finding the superstable orbits. Here is the code.

```
iterations = 20


mu = 3.25
n=3
N = 2**(n-1)
for i in range(iterations):
    x = 0.5
    dx = 0
    for i in range(N):
        x = mu * x * (1 - x)
        dx = x * (1 - x) + mu  * (1 - 2 * x) * dx
    mu = mu - (x - 0.5)/(dx)
    print(mu)
```

For some reason it didn't work out so well; I was only able to confirm the first superstable point $\mu = 2$. I suspect is has to do with finding appropriate initial values. I look forward to seeing what my classmates have done with this.

Another way that occurred to me as a way to find the superstable values of $\mu$ is using approximations of the Lyapunov exponent. One definition of the Lyapunov exponent is

$$\lambda = \lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} \log_2 \left| \frac{dx_{n+1}}{dx_n} \right|,$$

so one can find an approximation of $\lambda$ by simply fixing $N$, in which case we obtain

$$\lambda \approx \frac{1}{N} \sum_{n=1}^{N} \log_2 |\mu - 2\mu x_0|.$$

If we are able to find a sequence of partial sums of Lyapunov exponents that diverges to $-\infty$, then the corresponding parameter is superstable. Looking at the picture on the next page, you can clearly see asymptotes in the Lyapunov spectrum (denoted by the vertical blue lines). It is conceivable that one could select two points on either side, and use a sort of bisection method refine the approximation for $\mu$, thereby finding the desired convergent sequence.

Determining the superstable points this way is perhaps less complicated than Sternberg's method, although I have yet to work out the details.

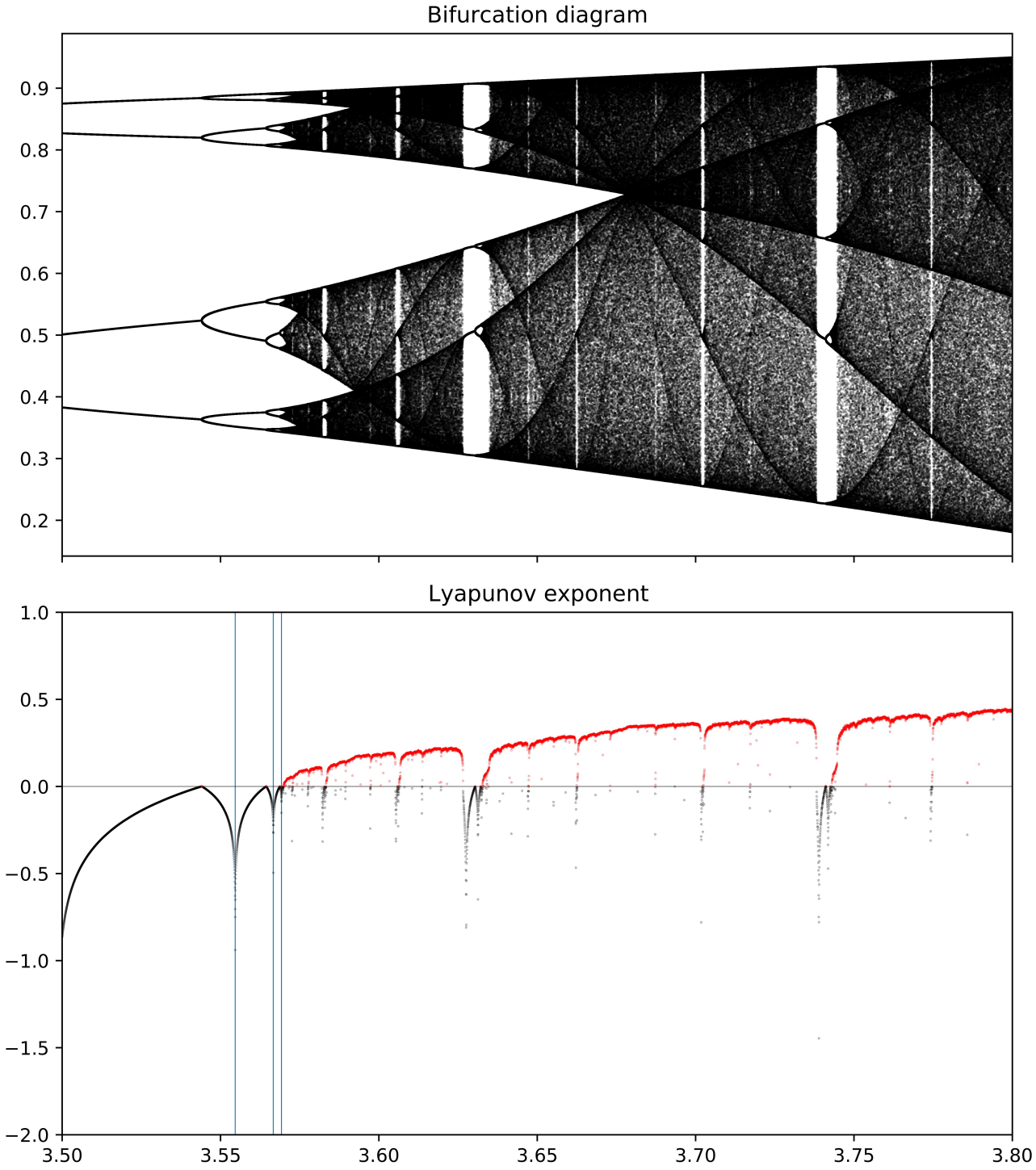



Figure 10: We expect the superstable orbits to occur at the apparent asymptotes of the Lyapunov exponent